

Request Cancellation in Web Application Frameworks

TUDOR ALEXANDRU MATEI, University of Twente, The Netherlands

Supervised by: dr. L. Ferreira Pires

This paper investigates how request cancellation in web application frameworks is influenced by their underlying architectures. Modern web applications need to consider that client requests can be terminated earlier than expected due to the user closing a tab, navigating away, losing connection, etc. Therefore, improper termination leaves the request computation redundant as the user is no longer present to receive the response. As a result, the server sustains unnecessary resource usage or produces undesirable behavior. This paper classifies web frameworks by the cancellation signal source, which influences the route a disconnect takes to running application code. This yields three mutually exclusive architecture classes: passive, cooperative, and reactive. The study compares the three classes by using identical measurement infrastructure and idiomatic implementations across three different candidate frameworks. Through empirical measurement, the study compares the lifecycle of detection, propagation, and termination of the cancellation signal. The passive architecture does not detect the signal by specification, while the reactive architecture is load-invariant, whereas the cooperative detection degrades under thread-pool saturation along with a predictable cliff. Furthermore, the abort protocols determine the verifiability of work stoppage, as the reactive framework is decoupled and cooperative cancellation is the termination event itself. Finally, incomplete cancellation for all architectures follows Little's law for resource occupancy, while state consistency is entirely dependent on the transaction boundary.

Additional Key Words and Phrases: request cancellation, web frameworks, concurrency models, cooperative cancellation, reactive streams

1 INTRODUCTION

Modern web applications are dependent on platforms to serve a large number of concurrent user requests. In real-world applications, many client requests are terminated before completion [10]. This can occur when a user closes a browser tab, navigates away from a page or loses network connection. When this happens, the connection between the client and the server is closed while the server may still be executing the associated request logic. Many web platforms implement specific request cancellation mechanisms driven by the framework architecture [4].

Request cancellation refers to the termination process of computations when tasks become redundant or fail. This study investigates cancellation triggered connection loss, which can be either user induced or due to client-side interruption. In high-capacity environments, once termination occurs, all redundant work must be canceled gracefully, as any work done from that point is effectively wasted. This leads to unnecessary resource usage, which can impact subsequent requests through increased delays and availability. Besides efficiency concerns, improper handling of cancellation can impact the consistency of data. Partially executed operations can produce unintended behavior that is invisible to the user.

The challenges introduced by request cancellation are further complicated by the different concurrency models used by the platforms. This makes it difficult to accurately choose the appropriate

platform for each use case. Many web application frameworks provide request cancellation mechanisms, however we could not find any common understanding on how these mechanisms are shaped by the underlying concurrency model on top of which they are built. Moreover, the concurrency model has a major impact on how the system developer can implement such mechanisms using idiomatic rules imposed by each framework. This results in differences in whether work actually stops, how quickly, and how far the signals propagate.

To make these differences measurable, this paper compares representative frameworks that span the different request cancellation architectures. The comparison is framed around the life cycle that a successful cancellation signal must follow: it must be *detected*, *propagated* through the request layers, and lastly redundant work should be *terminated*. This naturally leads to the following research question: *How do the request cancellation architectures of web application frameworks differ in their ability to detect, propagate, and terminate a client request, concerning reliability, resource utilization, and system state correctness?* The main question is answered by addressing the following research questions:

- RQ1.** How quickly and reliably does each framework detect a client disconnect signal under load?
- RQ2.** How does the cancellation signal propagate through system layers under each framework's idiomatic implementation?
- RQ3.** What are the behavioral consequences of incomplete cancellation propagation, in terms of resource occupancy and system state?

This paper answers the defined questions using empirical data gathered through controlled experiments, by comparing representative frameworks that span the cancellation architectures identified. The cancellations in these experiments were triggered through controlled disconnection signals, with precise timing management.

2 BACKGROUND

2.1 Request Cancellation

Request cancellation is the main topic of this paper. In this work, the concept of canceling requests refers to the server-side graceful termination of in-flight computations. In this paper we consider cancellation induced through planned *Transmission Control Protocol* (TCP) connection discontinuation. Furthermore, the cancellation signals studied have to be graceful, meaning that the signal is not allowed to force kill running code, but rather has to safely terminate the tasks at hand. This means that in the work being done the signal must be observed and acted upon voluntarily. Building on the phase breakdown proposed in [24], this paper introduces the general request cancellation lifecycle as three stages: detection, propagation and termination. These phases structure the paper's three sub-questions and aid in having a complete overview of the cancellation process.

TS&IT 45, July 3, 2026, Enschede, The Netherlands

© 2026 University of Twente, Faculty of Electrical Engineering, Mathematics and Computer Science.

The cancellation lifecycle can be more closely studied by looking at its underlying pipeline. The requests traverse a layered path: transport (socket) → application code → downstream resource. The downstream resource can represent a database, an outbound service call, a file read or similar. The cancellation signal originates in the transport layer, where the client closes the connection (a TCP FIN packet is sent), which the server must first notice before any cancellation can begin. The signal has to then travel each layer boundary, as any layer that does not forward the signal stops propagation leaving remaining work running unnecessarily.

2.2 Concurrency Models

The methods in which the cancellation signal is detected and propagated are shaped by the underlying concurrency model that is used by the server web framework. The concurrency model defines how threads and processes can communicate, share resources, and synchronize to execute multiple tasks simultaneously. Furthermore, it determines how it maps incoming requests onto execution context, so that many user requests can be served simultaneously. Several concurrency models are used in production: thread-per-request, event-driven, virtual threads, and actor model [8, 28]. However, for cancellation, this narrows down to two model classes based on which execution context owns the client request:

- **Thread-per-request** (blocking): every incoming request is assigned a worker thread for the duration of its lifetime [1]. This means that the thread assigned is blocked until the request is completed or canceled. The virtual thread model is a lightweight variant but with identical request ownership rules [19].
- **Event-Driven** (non-blocking): a small pool of threads (event-loop) handles request connections, rather than assigning one thread per connection, with all request work being offloaded off the event-loop onto a separate pool of worker threads [26].

2.3 Cancellation-propagation architectures

Concurrency models shape how request cancellation is implemented in modern web frameworks, but they do not determine how the disconnect signal reaches the application code. One argument for this statement is that some frameworks utilize a combination of concurrency models which behave differently. Therefore, we classify the cancellation architectures based on the signal source, meaning the route by which a disconnect signal reaches running application code. We classify architectures based on the signal because the signal source is the independent variable that governs behavior in this study, and is precisely what determines the answer to the research questions. Therefore, we have defined the following cancellation architecture taxonomy based on the signal source:

Passive: signal only appears as an error when the server tries to write back to the client and finds the connection closed. While the received request is computing or awaiting I/O there is no signal to be received, as no write-backs are attempted to the client, therefore, the work runs to completion regardless of whether the client is still present. This behavior is associated with the thread-per-request concurrency model, as each request gets assigned a thread which is

busy with the computation, while no separate process is watching the connection socket. The decisive factor, however, is the underlying server specification, which defines no disconnect notification. A thread-based framework is able to detect request cancellation given that the server specification surfaces the functionality. Popular web frameworks that follow this request cancellation architecture include: Spring MVC (the Servlet specification) [25], Django [7], Ruby on Rails [23], none of which define an API for surfacing a mid-request disconnect to application code.

Cooperative: signal is detected automatically and appears as an explicit token the developers must forward through the call stack and observe. This implies that detection is automatic (socket is being monitored) but propagation is manual (the signal only reaches the layers in which the token is passed through and checked). Therefore, any CPU work that blocks the current thread can be canceled provided the token is polled. This architecture is independent of the concurrency model as a token can be carried through thread-per-request or event-driven code alike. For this reason the cooperative cancellation architecture appears in both types of web application frameworks. Some examples include: ASP.NET Core (CancellationToken) [13], Go (context.Context) [29], and Node.js (AbortController) [17].

Reactive: work is expressed as a pipeline of operators [22]. These pipelines consist of a publisher that emits items, intermediate operators that transform them, and a subscriber that consumes them, all of which are connected by a subscription. The name “reactive” is representative of the way the subscriber consumes the items, as the subscriber controls the rate at which the publisher emits. Rate-limiting of data based on subscriber availability is called backpressure. In this architecture, cancellation is simply performed by the subscriber canceling its subscription to the publisher, which initiates an automatic propagation of the cancellation signal through all the operators in the pipeline until it gets to the publisher. Therefore, propagation is automatic and structural as opposed to the manually threaded approach of the cooperative model. This is characteristic of the event-driven non-blocking model on which reactive web frameworks are built. Some examples include: Spring WebFlux (Project Reactor) [27] and Vert.x [5].

Our cancellation taxonomy above corresponds to a decision tree. A disconnection signal can either reach the application code or not; when it does not, this characterizes the passive model; when it does, its propagation is either threaded manually (cooperative) or fully automatic (reactive). These distinctions yield exactly three architectures by construction.

3 LITERATURE REVIEW

Two methods were used to gather relevant literature sources for this paper. Google Scholar and IEEE Xplore were the basis for searching for specific keywords such as “request cancellation”, “concurrency models”, “task cancellation”, and “concurrent frameworks”. Furthermore, a secondary round of research was conducted with AI tools for keyword generation, after which every selected paper was thoroughly human-reviewed.

Previous research has been done on the performance of back-end web frameworks, concurrency models or even underlying web server architectures [3, 15, 18]. The studies that compare web frameworks often focus on throughput, latency and scalability under load [3, 18]. Furthermore, benchmarking research has analyzed how these metrics are impacted by the underlying concurrency model for both event-driven and thread-per-request architectures [18]. Studies comparing web frameworks classify them by performance characteristics [3] or by concurrency model [18].

Other studies focus on asynchronous programming design choices, such as futures, promises and reactive-streams [2, 16]. These studies look at how this execution model impacts developer code writing experience, error handling and system performance. This research is key in understanding how asynchronous programming is used in both thread-per-request and event-driven web frameworks. Furthermore, these concepts are necessary to define the reactive cancellation architecture present in the taxonomy.

Some studies address the cooperative cancellation mechanism directly. These studies model the cancellation action as an explicit signal, formally a cancellation token, which is tripped when cancellation is requested and needs to be passed down the call stack, polled periodically and acted upon to terminate computation voluntarily [9, 24].

Furthermore, there are studies that look at interrupting long running, heavy database queries in order to save resources under load [6]. Many database systems provide mechanisms to cancel ongoing queries; however, their effectiveness depends entirely on how the application layer propagates the signal to the database driver [11]. This means that if the application does not support request cancellation, then the database driver cannot make use of cancellation mechanisms. Furthermore, [11] treats the propagation of the cancellation signal as a property of the application layer, but does not examine whether the bridge, which in this case is the driver of the database, forwards the signal correctly. Propagation of signals can therefore break at any intermediary steps that mishandle it, not only at application layer, as each link in the chain either forwards the signal properly or silently absorbs it.

Recent work analyses the importance of well defined request cancellation behavior [24]. Some studies discuss how to define and implement request cancellation and what benefits it provides [11]. Qualitatively, the cancellation topic has been studied, as one can easily find how each web framework supports request cancellation and how this is implemented in practice through official documentation resources. However, request cancellation support is often reported as being binary: either successful or unable to cancel. To our knowledge, no prior work classifies web frameworks by how the client-disconnect signal reaches application code. This is the basis for the taxonomy we introduced in section 2.3.

Existing resources therefore establish that web frameworks support cancellation and show how to implement such systems; however, three gaps still remain. First, detection is represented as supported-or-not rather than being quantified under concurrent load. Second, propagation into downstream layers is inferred rather than measured. Third, the resource usage and data consistency consequences of incomplete propagation are not quantified. This measurable data can further aid in choosing the appropriate request cancellation

architecture for each use case, as quantitative data enables informed trade-offs.

4 METHODOLOGY

This paper presents a comparative, controlled study across the three cancellation-propagation architectures identified: passive, cooperative, and reactive. To investigate each architecture, a representative web application framework is chosen per category as an experimental candidate. Each candidate implements the best-effort idiomatic implementation of the testing scenario, ensuring full transparency of official documentation resources. The testing scenarios simulate possible application patterns: asynchronous operations, thread-blocking work, or outbound interactions. The candidate frameworks differ widely in terms of programming language, runtime, and concurrency model. Therefore, to make the comparison fair, the main goal is equivalence at a higher level of abstraction provided by the testing scenarios and measurement infrastructure.

4.1 Candidate frameworks

Table 1. Frameworks, architectures, and role in the study.

Framework	Architecture	Role
Spring MVC	Passive	Qualitative baseline
ASP.NET Core	Cooperative	Quantitative subject
Spring WebFlux	Reactive	Quantitative subject

The criteria used to choose the candidate frameworks in Table 1 are: industry popularity, documentation availability, and most importantly architectural representation of the cancellation signal. The popularity was determined based on 2025 StackOverflow and JetBrains surveys that show that Spring and .NET ecosystems are widely preferred for building enterprise-level web applications [8, 28]. As a result, the findings of this paper are directly applicable to a large sample of web application developers. Furthermore, the maturity of the Spring and .NET ecosystems means both are backed by extensive and actively maintained official documentation.

Most importantly, each framework is representative of a cancellation architecture. Spring MVC represents a passive architecture justified by the Servlet specification, which explicitly states no notification is provided when the client leaves, and the cancellation is detected only when write-back happens [25]. Similarly, ASP.NET Core represents the cooperative architecture, as Kestrel, the underlying web server, detects the client disconnect and exposes it as the `CancellationToken` [14]. The framework then raises the signal automatically but is dependent on the developer to propagate and poll the token appropriately [14]. Lastly, Spring WebFlux represents the reactive architecture. On client disconnect, the underlying web server, Netty, cancels the reactive subscription, and the cancellation propagates automatically upstream through the operator graph to the source publisher, without needing explicit developer wiring [21].

Table 1 shows that Spring MVC represents the qualitative baseline, as the Servlet specification guarantees 0% detection on non-streaming endpoints, so there is no variance to measure. Therefore, the main comparison happens between the cooperative and reactive architectures.

The quantitative comparison between ASP.NET Core (C#) and Spring WebFlux (Java) crosses a language and runtime boundary, which we acknowledge. This study does not attempt to normalize it away, but rather aims to raise the level of abstraction so that the results that are language-independent by construction, such as cancellation rate and ghost-write rate.

4.2 Experimental setup

To make the comparison fair, an abstract web application design has been defined and used in all experiments. The main structure is a monolith, each framework having implemented the following layers with their own responsibilities:

- *Controller layer*: Enables the REST API endpoints and forwards logic to service layer.
- *Service layer*: Implements the logic and is framework-agnostic.
- *Repository layer*: Handles database connections and interaction.

With this abstract design, each framework can be evaluated consistently, as implementation differences are separated by layers. For instance, one framework might have a different database driver, in which case only the repository layer has a significant deviation. This approach helps isolate varying components of the substantially different frameworks.

Cancellation of requests is induced, not simulated. In each experiment, the client issues a request to the server framework and then deliberately closes the underlying TCP connection at a planned wall-clock time, mid-request. To measure latency without network round-trip noise, the client embeds the planned disconnect time as a query parameter of the request and the server utilizes this client-supplied timestamp as a reference point for latency measurement.

Furthermore, a framework agnostic measurement device is defined, as measurements should not impact the experiments. Every action is logged to standard output stream using an identical event schema: a request arriving, a disconnect being detected, a cancellation reaching a given layer, and a query completing or aborting. After experiments are conducted, logs are gathered and analyzed, as the event logging schema is identical across frameworks.

The procedure of running the experiments rests on a set of experiment controls that aim to mitigate measurement noise. First, the entire experimental setup is dockerized, each external service and web framework implementation being fully isolated inside its own container. This container setup allows for a full container restart between experimental conditions to prevent carry-over state such as automatic thread-pool sizing, connection-pool states, or OS page caches. Furthermore, the isolation allows for runs to be kept independent, meaning that every run can monitor used services for each activity. For example, database experiments begin only once the database is verified to have no pending active queries from the previous runs. Second, by using a dockerized setup each service and framework implementation is assigned a fixed resource allocation. Docker allows for CPU and memory limits, which brings two clear advantages: the host is separated from experiments and services do not starve each other for resources. By preventing conflicts over resources, measurements contain no resource advantage. Furthermore, pinning each service to fixed version-controlled container

images makes the full experiments reproducible for future work or validation¹.

4.3 Metrics and validity

The metrics used for evaluating the cooperative and reactive request cancellation architecture are of three types: proportions, dimensionless ratios, and latencies.

Proportions are language independent by construction, which makes the comparison of ASP.NET and WebFlux reliable. The work relies on three main proportions: cancellation rate, per-run failure probabilities, and ghost-write rate. Cancellation rate refers to the fraction of requests where cancellation was detected, and work was stopped. Per-run failure probability is the run-level counterpart of the cancellation rate. Requests are sent as a burst of N concurrent disconnects per run, and a run is considered failed if the framework does not detect the disconnect for that burst. Thus, p_{fail} is the fraction of runs that fail. Finally, the ghost-write rate is the fraction of canceled requests that commit a database write that the client never acknowledged.

Dimensionless ratios are language independent as the units cancel leaving a numeric value. The main ratio used is the capacity ratio that represents the factor by which one architecture's maximum sustainable request rate exceeded the other's. Both quantities of the ratio represent request rates measured in an identical way, where the quotient removes the unit and language confound.

Latency metrics include raw millisecond measurements. First, detection latency is referred to as $L1$. This latency represents the time it takes for the framework to detect the disconnect minus the planned disconnect time supplied by the client. Second, hold time H represents the time period in which the canceled operation keeps running after the cancellation signal fires. This metric is computed by subtracting the operation's actual completion time from the cancellation time, and it represents the time for which the application assumes the work has stopped. Furthermore, it also represents the extra time the operation occupied system resources.

The latency metrics are prone to cross-language validity concerns such as differences in raw execution speeds, runtime compilation, and garbage collection behavior. However, the language scale mismatch and the load-dependence arguments address the validity issues. First, the scale mismatch refers to language-speed deltas for equivalent code implementations which are in the magnitude of nanoseconds, while most of our measurements have been in the scale of milliseconds. For instance, the $L1$ detection latency is dominated by interactions with the operating system, framework dispatch, and thread-pool scheduling, whereas the raw computation difference between C# and Java is a negligible fraction of this sum. Second, the load-dependence argument asserts that a language difference between frameworks is uniform across load. This means that the measured latencies will always be shifted by the same constant amount at every level of concurrency, this constant being dependent on the language raw computation differences. Therefore, by being load-invariant, the constant offset cannot account for any effect that varies with load.

¹The complete implementation, analysis and experimental results are available on GitHub at <https://github.com/tudormatei/request-cancellation-research>

4.4 Analysis

In each experiment, we repeated independent runs at a fixed configuration or combination of variables such as number of concurrent requests, available threads, cancel timings, scenario, etc. One such configuration is referred to as a cell. For normal cells, the sample size is 10 runs per cell, with this number being raised to 50 runs where stochasticity is the highest, so that more samples are required. Proportions are reported with Wilson 95% score intervals because the outcomes sit near the extremes: a fully detecting cell 10/10 gives a Wilson 95% interval of [72%, 100%] and a fully failing cell 0/10 gives [0%, 28%]. These two intervals are non-overlapping and separated by a substantial amount, meaning that the difference between a detecting and non-detecting configuration cannot be attributed to sampling noise.

Continuous measurements such as detection, propagation and hold times are expressed in terms of their mean and standard deviations or by median where distribution is skewed. Furthermore, trends among quantities, such as detection latency against concurrency are tested using Spearman’s rank correlation to express independence and test trends. Spearman is the appropriate tool to use as it does not assume linearity or normality of the data.

4.5 Experimental design

Each framework implements scenarios that replicate real-world computations done by web applications. The base scenarios implemented per framework are introduced below, and then mapped to the experiments and in-turn to the questions defined in section 1.

Table 2. Mapping of research questions to experiments and scenarios.

Question	Experiment	Scenario(s)
RQ1 Reliability	E1 (E1a)	CPU loop, Async
RQ1 Latency	E1 (E1b)	CPU loop, Async
RQ2 Propagation	E2	DB query, outbound HTTP
RQ3 Resources	E3	DB query under load
RQ3 State	E4	Single + two-step write

The scenarios in Table 2 span the layers a cancellation signal must cross (application code, the database, and outbound services). Application code is either blocking (CPU heavy task) or non-blocking (asynchronous task that releases the thread). The blocking task is polling for cancellation at fixed yielding points, where the yield intervals are set by the K parameter. The database is implemented using each framework’s idiomatic driver; namely Npgsql for ASP.NET and R2DBC for Spring WebFlux. The outbound layer implements both the HTTP/1.1 connection close abort and the HTTP/2 RST_STREAM cancellation call under the form of a gRPC call, to test both in-band and default TCP close cancellations.

The first experiment (E1) in Table 2 is split into two sub-experiments; E1a and E1b, with the aim of answering RQ1. E1a is the reliability experiment that investigates how request cancellation reliability behaves under load. In this experiment, we used the two defined scenarios: CPU-bound and asynchronous work, with varying N concurrent requests being canceled, while measuring the cancellation rate. Near the load N at which reliability breaks down, reliability is

described by the per-run failure probability, and the cliff location N_{safe} is taken as the smallest N at which this probability reaches 10%. Furthermore, to measure the cliff, N_{safe} is tested across a sweep of the yield interval K of the CPU-bound work and the CPU core count C , which are the two parameters that are expected to govern it.

E1b is a single-framework experiment, meaning that each cancellation architecture is measured independently. The experiment investigates detection latency of the cancellation signal and what governs it under load. Across the two previously defined scenarios (CPU-bound and asynchronous work), concurrency N , the maximum number of threads available T_{max} , and the yield interval K are varied, while keeping the framework, infrastructure, and request shape fixed. The measured variable is the $L1$ detection latency.

To measure RQ2, propagation depth of the cancellation signals inside the frameworks should be investigated. The second experiment (E2) analyses the process of the signal reaching the different layers present in the formulated scenarios. RQ2 does not only refer to which layers are reached (documentation derivable), but rather whether the signal actually crosses the I/O boundary to the application-external consumer. Three different protocols are studied to examine how the signal can cross the boundary: connection-close (outbound HTTP/1.1), in-band abort (outbound gRPC), and out-of-band abort (PostgreSQL query with CancelRequest on a separate connection). For each scenario, the main measurement is whether the signal is received at all by the application-external consumer.

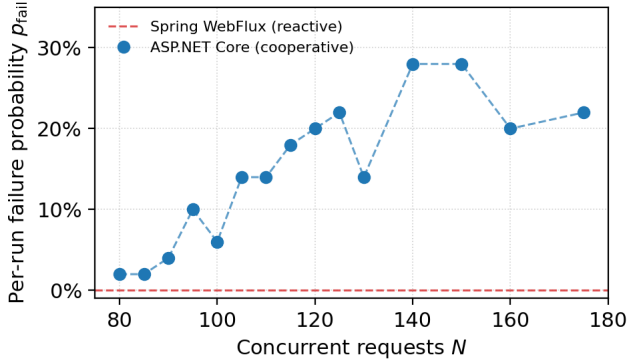
To answer RQ3, we performed experiments E3 and E4. E3 investigates the resource occupancy question that arises from incomplete request cancellation. The main scenario used is database querying under load: requests arrive at a rate X against a connection pool of size P , run for a fixed duration Q , and a fraction Y of them are canceled mid-flight, to simulate real-world request traffic. The database activity is measured using `pg_stat_activity` and analyzed in terms of the hold time H that represents the amount of time the resource connection was held after the cancellation signal appeared. Both X and P are varied across the value ranges [1, 20] and [1, 40] respectively. The maximum sustainable rate X_{max} per framework is determined at specific connection-pool sizes P to quantify the architectural consequences of incomplete cancellation.

Experiment E4 uses two scenarios: a single in-flight write and a multi-step write separated by a cancelable gap. The writes represent simple INSERT statements sent to the database. For single writes, D represents the duration of the commit transaction. For the multi-step write, the same two writes are run both with and without wrapping them in a database transaction. Every outcome is measured using the database directly, through the ghost-write rate metric. Each configuration is repeated over 20 trials.

5 RESULTS

5.1 Detection under load (E1)

E1a investigates the reliability of cancellation architectures. The passive architecture represented by Spring MVC reported a 0% cancellation rate for varying $N \in [1, 50]$ simultaneous requests, as no mechanism exists for detection. The reactive architecture (Spring WebFlux) reported a 100% cancellation rate in both scenarios, tested

Fig. 1. p_{fail} vs. concurrent requests N at $K = 100\text{ms}$, $C = 1$

up to $N = 300$ for the CPU-bound workload and $N = 750$ for the async workload. Under the async scenario, ASP.NET held 100% cancellation rate at all tested $N \in [1, 750]$. Conversely, for the CPU-bound scenario, the cooperative architecture (ASP.NET) in Figure 1 showed that beyond a specific N (the cliff), cancellation rate stops being 100% and is stochastic, with the per-run failure probability rising gradually from 2% at $N = 80$ to 28% at $N = 140$ ($K = 100\text{ms}$). We observed that the cliff location N_{safe} is well approximated by the following empirical function, where K represents the yield interval of the CPU-bound work and C the number of available CPU cores:

$$N_{\text{safe}} \approx 5499 \cdot \frac{C}{K_{\text{ms}}^{0.880}}$$

The empirical approximation predicts where the cliff N_{safe} is located for the cooperative model when only one CPU core is available. The formula's exponent of 0.880 has a 95% confidence interval of $[0.77, 0.99]$ and Pearson $r = -0.9959$ on the log-log regression, and was validated over $K \in [50, 500]\text{ms}$ with a maximum error of 11% (Table 3, Appendix). The value 5499 is the empirical scale constant, fitted as the regression intercept. Furthermore, C scaling was confirmed at $C = 2$ and $C = 4$.

E1b investigates the detection latency $L1$. The cooperative and reactive models both performed similarly in the asynchronous work scenario, with both measuring detection latency $L1$ in the sub-10ms range, for all N tested. For the CPU-bound work, WebFlux latency stays constant in the same range as asynchronous work, whereas ASP.NET is bimodal. The cooperative model measurement is either the same range as the async work (fast branch) or $2 \cdot K$ (slow branch), never in between. Which branch occurs is dependent on the worker-pool thread availability: with only one available thread $T_{\text{max}} = 1$ and one request $N = 1$, the slow $2K$ branch occurred 100% of runs (median 202 ms at $K = 100\text{ms}$), while with a spare worker ($T_{\text{max}} \geq 2$) the same request fell in the fast branch ($\approx 1\text{ms}$). In the slow branch, $L1_{\text{min}} = 2K$ was verified across $K \in [10, 500]\text{ms}$ with Pearson $r = 0.9998$ confirming $L1 \propto K$.

5.2 Propagation across the IO boundary (E2)

E2 investigates how the cancellation architectures can cross different IO boundaries. Application layer detection has been verified

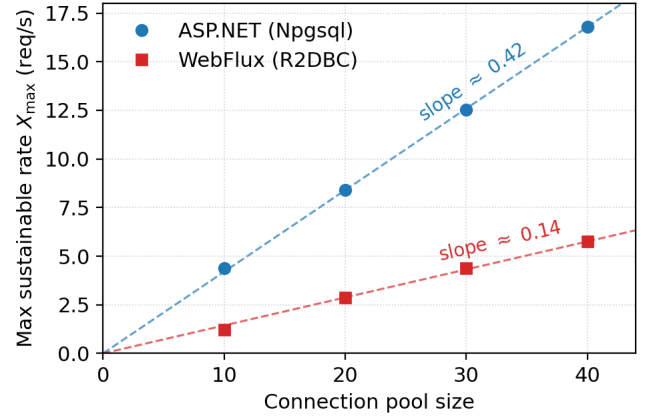


Fig. 2. Maximum sustainable request rate versus connection-pool size

in E1 with both cooperative and reactive detecting the signal, but passive did not. The main focus of E2 is therefore on the three possible abort protocols of external consumers, and how the signal is propagated. For the connection-close and in-band abort protocols, both frameworks detected and canceled 100% of the sent requests between 9 and 13 ms latency, confirmed by the downstream mock service. For the third protocol, out-of-band abort tested with PostgreSQL database, ASP.NET (Npgsql) stopped the query successfully and the database-cancellation rate equaled the detection rate at a latency between 40 and 120 ms. In contrast, WebFlux (R2DBC) sent no cancel signal, so every query ran to completion.

5.3 Resource occupancy under load (E3)

E3 investigates the consequences of incomplete cancellation of requests and how this affects the resource occupancy depending on the framework architecture. This experiment goes beyond E2, where WebFlux (R2DBC) did not cancel the queries it sent.

A canceled request's database connection remained occupied (its query active) on average for 54 ms after disconnect under ASP.NET, where the `CancelRequest` aborts the query, versus 4219 ms under WebFlux, where the query runs to completion. The measurements were done with an under-saturated database connection pool, with a 10 second total query duration Q and cancellation happening at the six second mark. When the connection pool was saturated, both frameworks failed to serve requests.

By varying the request rate X and database connection-pool size P (disconnect at 0.5 s, $Y = 0.8$), Figure 2 reports the maximum sustainable rate X_{max} for each architecture. Both scale linearly with the pool size, but ASP.NET sustains a much higher rate, namely a slope of 0.42 versus 0.14 for WebFlux.

5.4 Data state after cancellation (E4)

E4 investigates the data state each framework leaves after a request is canceled. For single in-flight writes, the ghost-write rate for WebFlux is 100%, whereas for ASP.NET this is dependent on the write's duration D relative to the disconnect time. However, in both cases, if a ghost-write happens the commits are application

silent, thus neither framework is notified of the commit. ASP.NET trials show when the INSERT committed before the cancellation was processed ($D < \text{the disconnect time}$) a ghost write occurred; when D is larger than disconnect time, then the `CancelRequest` aborted the INSERT and no row was written.

For multi-step writes comprised of two INSERTs with the client disconnecting in the gap between them, both ASP.NET and WebFlux committed one of the two rows (a torn write). If the multi-step is wrapped in a transaction, both frameworks roll back to zero rows with no connection left idle in the transaction.

6 DISCUSSION

6.1 Detection (RQ1)

The two detection experiments (E1a and E1b) expose one phenomenon. E1b's detection latency branches (≈ 1 ms or yield-bounded $\approx 2K$) together with E1a's stochastic cliff, define three points on a single saturation curve (fast $\rightarrow$ slow $\rightarrow$ fail). The curve is traversed as CPU-bound work rises and threads become unavailable. This happens because worker threads are busy with the CPU scenario computations, and the abort handler that is supposed to detect the disconnect signal does not get assigned a thread to work on. Therefore, cancellation detection is governed by whether the context that observes and delivers the signal can be starved by the request's own work. In WebFlux, the abort runs separately from the request work, so detection is flat and load-invariant, whereas ASP.NET picks a thread from the same pool of worker threads that requests use to detect abort signal. Therefore, the slow branch is a penalty that happens when ASP.NET has no spare workers. Given a free worker, ASP.NET detects as fast as WebFlux in ≈ 1 ms. Thus, the cooperative model is slower only when saturated.

The failing point of the curve happens at the cliff described by E1a, when the request abort is starved past the request's remaining life, thus detection fails. Past the cliff, detection becomes probabilistic, mainly because detected requests stay fast (either fast or slow branch), while the undetectable ones have no latency. The location empirical approximation N_{safe} , predicts the safe number of concurrent connections that can be handled based on two deployment parameters, namely CPU cores and yielding interval. By increasing cores or decreasing yielding interval, one can push the cliff outwards, handling successful cancellation for more requests. The formula and the parameters are specific to the .NET scheduler, but a second runtime written in Go confirmed that the cliff does exist for other cooperative frameworks and a similar approximation function can be derived.

For practitioners developing web application platforms, having the knowledge of the cliff in cooperative architectures is important when building a CPU-bound endpoint. As the endpoint has a computable safe ceiling, operating below the N_{safe} margin ensures detection is reliable and in turn quick. This means that load-invariant detection can be engineered rather than assumed, by adding more cores or by modifying the polling intervals for the tokens.

6.2 Propagation (RQ2)

E2 shows that both architectures cancel across two of the three boundaries, and the reactive one fails at the third. Both models try to

determine whether the cancellation reaches the consumer with the same vocabulary but different meanings. In the cooperative model the cancellation is a *completion event*: the handler fires because the database stopped. PostgreSQL aborts the query, returns error 57014, and Npgsql surfaces it as an `OperationCanceledException` wrapping a `PostgresException` with `SqlState="57014"`. This represents an exception that cannot exist unless the server has already stopped. The signal is therefore the physical proof of the server-side cancellation. In the reactive model, the cancellation is a notification decoupled from I/O state, as `doOnCancel` fires when the Reactor subscription is canceled, not when the server confirms it stopped. This is the central RQ2 finding: an asymmetry of verifiability, not of propagation reach.

Both architectures cancel correctly for connection-close and in-band abort protocols, but out-of-band abort fails in the case of the reactive framework. This is the case where the cancellation signal must travel on a separate connection and name which query to stop. This gap is specific to out-of-band cancellation protocols, of which the PostgreSQL `CancelRequest` protocol is one example and the one that was tested. The gap is caused by the architectural decisions, not an implementation defect of the database driver R2DBC. A reactive driver (such as this database driver) cannot safely issue an out-of-band abort such as `CancelRequest`, because of two defining reactive properties that forbid it: pipelining places many queries in flight on a single connection, so the driver cannot know which one the server is currently executing and backpressure forbids draining the pending responses to find out. In theory, a driver could solve this gap by introducing a non-pipelining mode, that is, by ceasing to be reactive. This is documented under an R2DBC GitHub issue #251 [20] requesting this exact gap fix. The maintainer stated clearly that the only way cancellation could work is by introducing a non-pipelining mode and so the issue was declined.

The practical implications of this experiment are that the cooperative signal reaches only the layers where the developer threaded the `CancellationToken`, and is silent wherever one was forgotten, whereas reactive cancellation propagates through the entire operator graph automatically. By combining this with the asymmetry finding, the cooperative and reactive architectures create a mirror image: cooperative is easy to verify, but easy to under-propagate, while reactive is easy to propagate, but impossible to verify from the application.

The experiment has two main threats to validity. First, only the idiomatic database drivers were tested together with only one database, namely PostgreSQL. Therefore, the reactive structural gap for out-of-band abort rests on the one driver measurement. However, the threat is defeated by arguing that the abort protocol is independent of the database management service and can be applied to any service that implements it. Second, the in-band and connection-close abort protocols are simulated with a mock downstream service that holds the request open for a fixed delay rather than performing real work. Therefore, we did not measure whether the downstream halts its computation, but rather the cancellation signal traveling and respecting the abort protocol. Whether the consumer acts on this signal is a downstream concern, not a propagation one.

6.3 Consequences (RQ3)

RQ2 establishes where a cancellation signal reaches, whereas RQ3 investigates what follows when it does not. Therefore, E3 and E4 tested resource occupancy and data state, respectively.

E3 shows that a canceled request occupies the database connection pool for the hold time H , which in a stable pool follows Little's law [12].

$$X_{\max} = P/E[S] \text{ with } E[S] = (1 - Y) \cdot Q + Y \cdot H$$

Where Y is the ratio of requests canceled, Q is the total duration of the uncanceled query, and P is the database pool size. The linear scaling in Figure 2 shows that this law applies, and that the two slopes are representative of each architecture's hold time H . Both architectures behave identically until disconnect and differ only afterward. Cancellation saves only the post-disconnect tail, so ASP.NET's advantage shrinks as disconnects occur later. This is shown by bounding the advantage with the ratio between ASP.NET and WebFlux from $1/(1 - Y)$ at an instant disconnect ($H = 0$) down to $1 \times$ when the client waits until the query nearly finishes ($H \rightarrow Q$).

Furthermore, this finding is important because it is independent of the database. Little's law holds for any bounded pool as a request acquires a resource on entry and releases it when it finishes. This applies for worker threads, semaphore permits, outbound connection pools, file handles, etc. Hold time H can represent any resource, such as a reactive driver not canceling due to out-of-band abort protocol, or a cooperative handler whose developer forgot to thread the CancellationToken into one call. Both scenarios produce the same effect which influences resource capacity.

E4 investigates ghost-writes that are induced by incorrect cancellation. The results showed that WebFlux produced one every time despite the cancel, so any in-flight write commits unconditionally. ASP.NET protects against this as long as the disconnect happens before the write query finishes. In practice, real writes commit in milliseconds, while clients disconnect seconds later, therefore, both frameworks are prone to ghost-writes. The important point is that the ghost-write is silent, meaning that the commit happens before cancellation, and so does not raise a cancellation exception. With no exception, the application assumes that the write was aborted, and so only a database level audit can confirm it. Therefore, the decisive result of E4 is that the cancellation architecture does not govern state consistency, but rather the transaction boundary does. A real operation usually involves several writes, so the two step write scenario tests the cancellation gap in the middle. The results show that without wrapping the whole chain of writes in a transaction, the write is left torn (step one committed and step two never ran) under both ASP.NET and WebFlux. In the case of transactions, both frameworks are able to roll back safely and preserve data integrity. The passive Spring MVC architecture is the only one that leaves consistent data, because it never detects the disconnect and runs both writes to completion. Therefore, honoring cancellation without a transaction is worse for state integrity than ignoring it. This resolves an apparent contradiction with RQ2, as WebFlux is not able to abort a query, however, inside a transaction, it still rolls back cleanly. This does not mean the query computation is stopped,

but rather the query is never committed, because the reactive subscription's cancellation triggers the transaction operator's rollback of the uncommitted row.

The two experiments have two main threats to validity that need to be addressed. First, only one instance of the capacity law was directly measured, namely the database connection pool. By extension, other bounded pools (worker threads, semaphore permits, file handles) and other causes of the hold time follow the same queuing argument provided by Little's law. Second, for the multi-step write the measurement is the result of a single disconnect landing in the gap between the two writes rather than against every possible combination, but this would only give the same discrete outcomes.

7 CONCLUSION

This paper compares three request cancellation architectures: passive, cooperative, and reactive using candidate web frameworks, to observe detection, propagation, and termination of the cancellation signal. The main finding is that the architectures do not differ along a single "better-to-worse" axis, but rather present clear trade-offs.

Detection (RQ1). Detection reliability is determined by whether the process observing the disconnect signal, can be starved by the request's own work. Reactive detection is load-invariant whereas the cooperative matches it under thread availability from a latency and reliability point of view, but degrades significantly when the thread-pool is saturated. The saturation leads to an increase in detection latency before an outright detection cliff. The location of the cliff is predictable and dependent on application parameters. Passive frameworks detect nothing by design.

Propagation (RQ2). Both the cooperative and reactive propagate the cancellation signal through application layers, but differ when crossing into outbound service communication. Both architectures propagate successfully across connection-close and in-band abort protocols, and diverge only at out-of-band abort, where reactive cannot cancel. This gap is structural to the reactive architecture, as the cooperative cancellation represents a completion event, whereas reactive cancellation is only a notification, decoupled from the I/O state. Furthermore, the cooperative architecture needs manual cancellation token wiring and checking, whereas reactive is automatic.

Consequences (RQ3). Incomplete cancellation has consequences in resources and data state. Such an incomplete request, shows a reduction in sustainable load by showing that any bounded pool caps throughput by Little's law. The difference in throughput between cooperative and reactive is shown to be dependent on the time saved by the cancellation, which impacts throughput significantly. In contrast, data state consistency is not determined by the architecture at all but by the transaction boundary.

Future work. One could perform the experiments on more frameworks and classify them according to the taxonomy. By having more evidence, this step would allow industry developers to make better choices regarding framework selection. Furthermore, one could test and confirm the capacity law on other bounded pools like worker threads, semaphore permits, or outbound connections, which would confirm the generalization made.

REFERENCES

- [1] Apache Software Foundation. 2024. Apache Tomcat 8 Configuration Reference: The HTTP Connector. Apache Tomcat Documentation. <https://tomcat.apache.org/tomcat-8.5-doc/config/http.html>
- [2] Oleksandr Serhiyovych Blahodelskyi. 2024. Asynchronous Programming in Javascript: Modern Approaches and Practice. *American Research Journal of Computer Science and Information Technology* 7, 1 (2024), 28–32. <https://doi.org/10.21694/2572-2921.24006>
- [3] Imad Buljic, Esad Kadusic, Tamara Cvijanovic, Narcisa Hadzajlic, and Natasa Zivic. 2025. Comparative Performance Analysis of Leading Backend Frameworks for Developers. In *2025 24th International Symposium INFOTEH-JAHORINA (INFOTEH)*. 1–5. <https://doi.org/10.1109/INFOTEH64129.2025.10959250>
- [4] Mila Dalla Preda, Maurizio Gabbriellini, Ivan Lanese, Jacopo Mauro, and Gianluigi Zavattaro. 2011. Graceful Interruption of Request-Response Service Interactions. In *Service-Oriented Computing*. Gerti Kappel, Zakaria Maamar, and Hamid R. Motahari-Nezhad (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 590–600.
- [5] Eclipse Vert.x. 2026. Vert.x Core Manual. Vert.x Documentation. <https://vertx.io/docs/vertx-core/java/>
- [6] Michael J. Franklin et al. 2007. Stop-and-restart Style Execution for Long Running Decision Support Queries. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB 2007)*.
- [7] Carlton Gibson. 2024. Pull Request #431: Should we specify that the exception MUST have a code attribute? Django asgiref GitHub repository, maintainer comment. <https://github.com/django/asgiref/pull/431>
- [8] JetBrains. 2025. The State of Developer Ecosystem 2025: Tools and Trends. <https://devecosystem-2025.jetbrains.com/tools-and-trends>. Accessed: 2026-04-26.
- [9] A. Kolesnichenko, Sebastian Nanz, and B. Meyer. 2013. How to Cancel a Task. In *International Conference on Multicore Software Engineering, Performance, and Tools*. https://doi.org/10.1007/978-3-642-39955-8_6
- [10] E. Kraemer, G. Paixão, D. Guedes, W. Meira, and V. Almeida. 2000. Minimizing the impact of orphan requests in e-commerce services. *SIGMETRICS Perform. Eval. Rev.* 28, 2 (Sept. 2000), 36–42. <https://doi.org/10.1145/362883.362911>
- [11] Abhinaw Kumar. 2025. Handle Request Cancellation in ASP.NET Core. <https://bytecraft.dev/posts/aspnet-core/handling-request-cancellation/>. Accessed: 2026-04-25.
- [12] John D. C. Little. 1961. A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research* 9, 3 (1961), 383–387. <https://doi.org/10.1287/opre.9.3.383>
- [13] Microsoft. 2026. CancellationToken Struct. Microsoft Learn, .NET API Reference. <https://learn.microsoft.com/en-us/dotnet/api/system.threading.cancellationtoken>
- [14] Microsoft. 2026. Task Cancellation. <https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/task-cancellation>. Accessed: 2026-06-05.
- [15] Van Nam Nguyen. 2017. Comparative Performance Evaluation of Web Servers. *VNU Journal of Science: Computer Science and Communication Engineering* 31, 3 (2017), 28–34.
- [16] Vu Thien Nga Nguyen. 2015. *An Efficient Execution Model for Reactive Stream Programs*. Ph.D. Dissertation. University of Hertfordshire, Hatfield, UK. <https://doi.org/10.18745/th.16361>
- [17] Node.js Foundation. 2026. Global Objects: Class: AbortController. Node.js API Documentation. <https://nodejs.org/api/globals.html#class-abortcontroller>
- [18] David Pariag, Tim Brecht, Ashif Harji, Peter Buhr, Amol Shukla, and David R. Cheriton. 2007. Comparing the performance of web server architectures. In *Proceedings of the 2nd ACM SIGOPS/EuroSys European Conference on Computer Systems 2007 (Lisbon, Portugal) (EuroSys '07)*. Association for Computing Machinery, New York, NY, USA, 231–243. <https://doi.org/10.1145/1272996.1273021>
- [19] Ron Pressler and Alan Bateman. 2023. JEP 444: Virtual Threads. OpenJDK. <https://openjdk.org/jeps/444> JDK Enhancement Proposal, finalized in JDK 21.
- [20] r2dbc-postgresql contributors. 2020. Support for cancelling a running query (statement cancellation). GitHub issue #251, pgjdbc/r2dbc-postgresql. Opened 2020; closed January 2022 as status: declined. Available at <https://github.com/pgjdbc/r2dbc-postgresql/issues/251>, accessed 2026-06-10.
- [21] Reactive Streams. 2026. Reactive Streams JVM. <https://github.com/reactive-streams/reactive-streams-jvm/blob/master/README.md>. Accessed: 2026-06-05.
- [22] Reactive Streams Special Interest Group. 2026. Reactive Streams Specification for the JVM. GitHub. <https://github.com/reactive-streams/reactive-streams-jvm>
- [23] Ruby on Rails. 2026. ActionController::Live::ClientDisconnected. Ruby on Rails API Documentation. <https://edgeapi.rubyonrails.org/classes/ActionController/Live/ClientDisconnected.html>
- [24] Utsav Sethi, Haochen Pan, Shan Lu, Madanlal Musuvathi, and Suman Nath. 2022. Cancellation in Systems: An Empirical Study of Task Cancellation Patterns and Failures. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, Carlsbad, CA, 127–141. <https://www.usenix.org/conference/osdi22/presentation/sethi>
- [25] Spring Framework. 2026. Asynchronous Requests. <https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-async.html>. Accessed: 2026-06-05.
- [26] Spring Framework. 2026. Spring WebFlux Overview. Spring Framework Reference Documentation. <https://docs.spring.io/spring-framework/reference/web/webflux/new-framework.html>
- [27] Spring Framework. 2026. Spring WebFlux Overview. Spring Framework Reference Documentation. <https://docs.spring.io/spring-framework/reference/web/webflux/new-framework.html>
- [28] Stack Overflow. 2025. 2025 Developer Survey: Technology Trends. <https://survey.stackoverflow.co/2025/technology/>. Accessed: 2026-04-26.
- [29] The Go Authors. 2026. Package context. Go Standard Library Documentation. <https://pkg.go.dev/context>

APPENDIX

AI Statement

During the preparation of this work, I used the following AI tools: ChatGPT (OpenAI) was used during the literature review phase to generate search keywords, which were used to query paper archives and identify relevant citations. It was also used for grammar correction and sentence clarity improvements in the written text. Claude Code (Anthropic) was used to assist in writing experiment runner scripts, data collection scripts, and analysis scripts across all sub-research questions. All AI-generated content was thoroughly reviewed and edited by the author. Full responsibility for the accuracy and integrity of this work rests with the author.

Cliff-location fit validation

Table 3. Observed cliff location vs. the empirical function for $C = 1$

K_{ms}	N_{safe} observed	N_{safe} predicted	Error
50	175	176	0%
100	95	95	0%
150	66	67	−1%
200	55	52	+6%
400	25	28	−11%
500	25	23	+8%